Process Scheduling Simulator — Project Documentation

Table of Contents

1. Project Overview
2. Program Design and Architecture
3. Module Descriptions
4. User Manual
5. Unit Testing
6. Use of Assertions
7. Team Contributions

---

1. Project Overview

This project implements an **event-driven process scheduling simulator** for an operating system. The simulator models:

- A multi-processor system using **preemptive Round-Robin** scheduling
- A high-priority **system process** that handles system calls periodically
- **Virtual memory management** with an **LRU (Least Recently Used)** replacement policy and disk transfer simulation

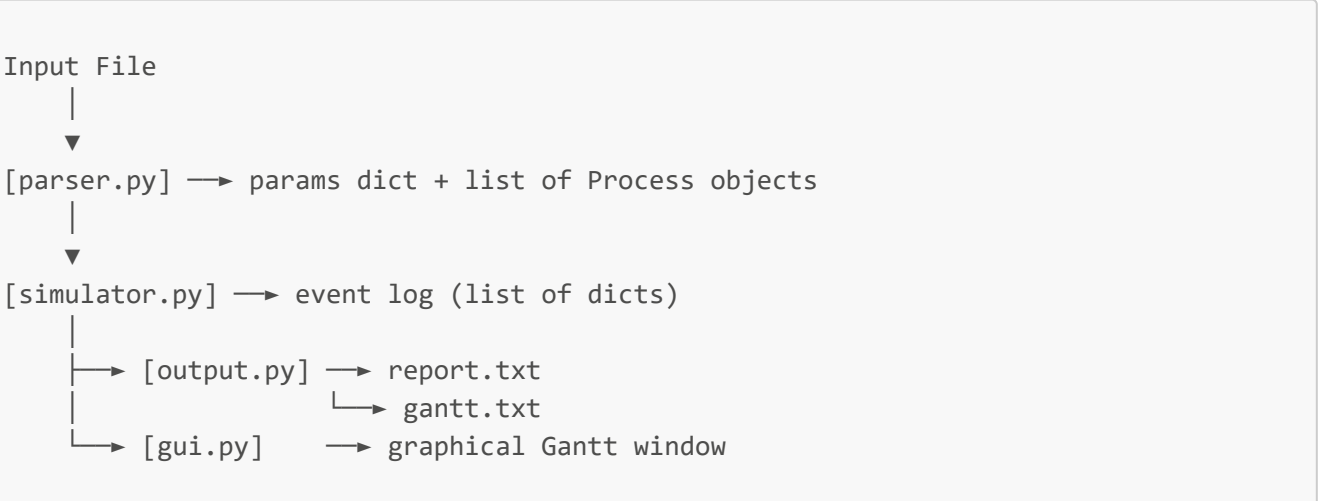
The simulator reads a configuration file, runs the simulation, and produces two outputs: a **text event log** and a **Gantt chart** (both ASCII and graphical).

---

2. Program Design and Architecture

2.1 Design Principles

The program is structured as a **pipeline of independent modules**:



Each module has a single, well-defined responsibility. The `models.py` module provides the shared data structures used throughout.

2.2 Event-Driven Simulation

The simulator uses a **sorted event queue** (`_events`). At each step, the earliest event is popped and processed. Events are:

Event	Description
`PROCESS_RELEASE`	A user process becomes available
`SYS_RELEASE`	The system process is released periodically
`CPU_DONE`	A time-slice or burst has finished on a processor
`SYSCALL_DONE`	The system process has finished a system call
`MEM_READY`	A disk-to-RAM transfer has completed

After every event, the scheduler first tries to dispatch any pending system calls (`_try_run_syscall`), then assigns ready user processes to free processors (`_schedule`).

2.3 Scheduling Logic

- **Round-Robin**: each process runs for at most `time_slice` units before being preempted and placed back at the end of the ready queue.
- **Processor affinity**: a process is preferentially scheduled on the processor it last ran on, if that processor is free.
- **System process priority**: the system process always gets a free CPU before any user process.

2.4 Virtual Memory

- The disk is a **serial resource**, only one transfer (SWAP_IN or SWAP_OUT) can happen at a time.
- When a process needs to run but is not in RAM, other processes are evicted using the **LRU policy** until there is enough space.
- Eviction only picks processes that are not actively running (`RUNNING`, `SYSCALL`, or `WAITING_MEM` states are protected).

---

3. Module Descriptions

`models.py` - Data Models

Defines the core data structures shared across all modules.

`Process`

Attribute	Description
`pid`	Unique process identifier
`release_time`	Time at which the process enters the system

Attribute	Description
`memory_required`	Amount of RAM needed
`bursts`	List of CPU burst durations
`syscall_times`	List of system-call durations (one between each pair of bursts)
`burst_index`	Index of the current burst
`burst_remaining`	Time remaining in the current burst
`state`	One of: `NEW`, `READY`, `RUNNING`, `SYSCALL`, `WAITING_MEM`, `DONE`
`in_memory`	Whether the process is currently loaded in RAM
`last_processor`	ID of the processor it last ran on (for affinity)

`Processor`

Represents a CPU. Tracks what it is currently running and when it becomes free (`busy_until`).

`SystemProcess`

Represents the high-priority system process. Tracks its release period, current state (`IDLE`, `WAITING`, `RUNNING`), and the queue of pending system calls.

`parser.py` - Input Parser

Reads the simulation input file and constructs a `params` dictionary and a list of `Process` objects.

Input file format:

```
PROCESSORS <n>
MEMORY <ram>
TIME_SLICE <q>
SYSCALL_PERIOD <p>
DISK_TRANSFER_RATE <r>

PROCESS <pid> <release_time> <memory>
BURSTS <count> b0 [s0 b1 s1 ... bn-1]
```

- Lines starting with `#` are treated as comments.
- The `BURSTS` line uses interleaved format: burst, syscall, burst, syscall, ..., burst.
- All five global parameters are required. The parser raises `AssertionError` if any are missing.

`memory.py` - Memory Manager

Manages RAM using the LRU replacement policy.

Method	Description
`complete_load(process)`	Marks a process as loaded into RAM after a disk transfer
`touch(process)`	Moves a process to the MRU end of the LRU list
`evict_lru()`	Evicts the least recently used evictable process; raises `RuntimeError` if none are safe to evict

The internal `_in_memory` list is ordered from LRU (index 0) to MRU (last index).

`simulator.py` - Event-Driven Simulator

The core simulation engine. Initialised with `params` and a list of `Process` objects.

Key methods:

Method	Description
`run()`	Main loop — processes events until all processes are done
`_schedule()`	Assigns READY processes to free processors
`_try_run_syscall()`	Dispatches the next pending system call if a CPU is free
`_start_load(process)`	Initiates a disk-to-RAM transfer, evicting as needed

`run()` returns the complete `event_log` — a list of dicts, each describing one simulation event with `time`, `end_time`, `type`, and additional fields depending on event type.

`output.py` - Output Writer

Produces two output files from the event log.

Function	Output
`write_text_report(event_log, path)`	A formatted text table with one row per event
`write_gantt(event_log, processors_count, path)`	An ASCII Gantt chart with one row per processor

Gantt chart legend:

Symbol	Meaning
digit	Process id running on that CPU
`S`	System call being executed
`M`	Memory transfer (SWAP_IN or SWAP_OUT)
`.`	Idle

`gui.py` - Graphical Gantt Chart

Renders the event log as a scrollable, colour-coded Gantt chart using **tkinter** (Python standard library — the only third-party/library exception permitted by the requirements). Each processor has its own row; each process is assigned a distinct colour. System calls appear in red; memory transfers in light blue.

`main.py` - Entry Point

Orchestrates the full pipeline: parse → simulate → write outputs → (optionally) show GUI.

---

4. User Manual

4.1 Requirements

- Python 3.10 or later
- No external packages required (tkinter is included with standard Python)

4.2 Running the Simulator

```
python main.py <input_file> [--no-gui]
```

Examples:

```
python main.py example_input.txt
python main.py example_input.txt --no-gui
```

- Without `--no-gui`, a graphical Gantt chart window opens after the simulation.
- With `--no-gui`, only the text outputs are written.

4.3 Output Files

File	Description
`report.txt`	Full event log, one line per event, with timestamps
`gantt.txt`	ASCII Gantt chart showing processor activity over time

Both files are written to the **current working directory**.

4.4 Input File Format

```
# Comments start with #
PROCESSORS 2
MEMORY 100
TIME_SLICE 3
```

```
SYSCALL_PERIOD 10
DISK_TRANSFER_RATE 10

PROCESS 1 0 40
BURSTS 4 5 2 3 4 9 4 6

PROCESS 2 2 30
BURSTS 3 4 3 7 2 5
```

Parameter descriptions:

Parameter	Description
`PROCESSORS`	Number of CPUs (integer ≥ 1)
`MEMORY`	Total RAM available (numeric)
`TIME_SLICE`	Round-Robin time slice duration
`SYSCALL_PERIOD`	Period at which the system process is released
`DISK_TRANSFER_RATE`	Units of memory transferred per unit of time

`PROCESS` line: `PROCESS <pid> <release_time> <memory_required>`

`BURSTS` line: `BURSTS <count> b0 s0 b1 s1 ... b(count-1)`

- `count` burst durations interleaved with `count-1` syscall durations.

4.5 Running the Tests

```
python -m unittest discover -s tests -v
```

---

5. Unit Testing

Unit tests are located in the `tests/` directory and use Python's built-in `unittest` framework. Each module is tested independently; where a module depends on another, a **fake (mock) object** is used to isolate it.

5.1 Test Files

File	Module tested
`test_models.py`	`models.py`
`test_memory.py`	`memory.py`
`test_parser.py`	`parser.py`
`test_output.py`	`output.py`
`test_simulator.py`	`simulator.py`

5.2 Mocking

In `test_simulator.py`, a `FakeMemoryManager` class replaces the real `MemoryManager`. It replicates the interface (`touch`, `evict_lru`, `complete_load`) without any real disk I/O, allowing the `Simulator` to be tested in complete isolation.

5.3 Test Summary

`test_models.py` - 4 tests

Test	What is verified
`test_initial_state`	Correct default values on `Process` construction
`test_is_done_flag`	`is_done()` returns `True` only when state is `"DONE"`
`test_repr_contains_pid`	`repr()` includes the process pid
`test_process_with_empty_bursts`	Invalid input: `Process` with no bursts defaults `burst_remaining` to `0`

`test_memory.py` - 6 tests

Test	What is verified
`test_complete_load_marks_process_in_memory`	`in_memory`, `used_ram`, and `_in_memory` are updated after a load
`test_touch_moves_process_to_mru`	`touch()` moves a process to the MRU end of the list
`test_evict_lru_returns_oldest_nonbusy`	LRU eviction skips busy processes and picks the oldest safe one
`test_evict_lru_raises_when_no_safe_process`	Invalid input: `RuntimeError` raised when no process can be evicted
`test_complete_load_same_process_twice_doubles_ram`	Invalid input: calling `complete_load` twice on the same object doubles RAM usage
`test_touch_process_not_in_memory_is_noop`	Invalid input: `touch()` on an unloaded process is a silent no-op

`test_parser.py` - 5 tests

Test	What is verified
`test_parse_file_returns_parameters_and_processes`	A valid file produces correct params and `Process` objects
`test_missing_required_parameter_raises`	Invalid input: missing `SYSCALL_PERIOD` raises `AssertionError`

Test	What is verified
`test_bursts_without_process_raises`	Invalid input: `BURSTS` before `PROCESS` raises `AssertionError`
`test_empty_file_raises`	Invalid input: empty file raises `AssertionError`
`test_garbage_file_raises`	Invalid input: unrecognised keywords raise `AssertionError`

`test_output.py` - 2 tests

Test	What is verified
`test_write_text_report_contains_each_event`	Report file contains column headers and event types
`test_write_gantt_produces_processor_rows`	Gantt file contains processor rows, legend, and correct symbols

`test_simulator.py` - 5 tests

Test	What is verified
`test_try_run_syscall_schedules_syscall_if_cpu_free`	System call is dispatched when sys-proc is WAITING and a CPU is free
`test_start_load_returns_false_with_no_safe_evictions`	Invalid input: returns `False` when RAM is full with no evictable processes
`test_start_load_evicts_and_schedules_mem_ready`	Eviction and SWAP_IN are logged; `MEM_READY` is queued at the correct time
`test_schedule_assigns_ready_process_to_free_processor`	A READY process is dispatched to a free CPU
`test_run_completes_process_with_fake_memory_manager`	Full end-to-end run produces a `DONE` event and ends with `SIMULATION_END`

5.4 Running the Tests

```
python -m unittest discover -s tests -v
```

Expected result: **27 tests, 0 failures**.

---

6. Use of Assertions

Assertions are inserted directly into the application code (not in the tests) to enforce **preconditions**, **postconditions**, and **invariants** at runtime. They cause the program to fail immediately and clearly at the point where a contract is violated, rather than producing silent wrong results later in the simulation.

6.1 `models.py`

Location	Type	Assertion
`Process.__init__`	Precondition	`pid` must be a non-negative integer
`Process.__init__`	Precondition	`release_time` must be non-negative
`Process.__init__`	Precondition	`memory_required` must be positive
`Process.__init__`	Precondition	`bursts` must be a list or tuple
`Process.__init__`	Precondition	`syscall_times` must be a list or tuple
`Process.__init__`	Precondition	`len(syscall_times)` must equal `max(0, len(bursts) - 1)`
`Processor.is_free`	Precondition	`current_time` must be non-negative

6.2 `memory.py`

Location	Type	Assertion
`MemoryManager.__init__`	Precondition	`total_ram` must be positive
`MemoryManager.__init__`	Precondition	`disk_transfer_rate` must be positive
`touch`	Precondition	`process` must not be `None`
`evict_lru`	Precondition	`_in_memory` must not be empty before eviction is attempted
`evict_lru`	Postcondition	`used_ram` must not go negative after an eviction
`complete_load`	Precondition	The process's memory footprint plus current usage must not exceed total RAM

6.3 `simulator.py`

Location	Type	Assertion
`Simulator.__init__`	Precondition	`time_slice` must be positive
`Simulator.__init__`	Precondition	At least one processor must exist
`Simulator.__init__`	Precondition	Total RAM must be positive
`Simulator.__init__`	Precondition	`processes` must be a list
`_push`	Precondition	Event type must be a string
`_start_load`	Precondition	The process must not already be in memory

Location	Type	Assertion
`_start_load`	Precondition	The process must request a positive amount of memory

6.4 `parser.py`

Location	Type	Assertion
`parse_file` entry	Precondition	`path` must be a string
`parse_file` entry	Precondition	The file must exist on disk
`BURSTS` parsing	Precondition	A `PROCESS` line must precede any `BURSTS` line
`_validate_params`	Postcondition	All five required parameters must be present in the parsed result
`_validate_params`	Postcondition	`processors` ≥ 1
`_validate_params`	Postcondition	`memory` > 0
`_validate_params`	Postcondition	`time_slice` > 0
`_validate_params`	Postcondition	`syscall_period` > 0
`_validate_params`	Postcondition	`disk_transfer_rate` > 0

6.5 `output.py`

Location	Type	Assertion
`write_text_report`	Precondition	`event_log` must be a list
`write_text_report`	Precondition	`path` must be a non-empty string
`write_gantt`	Precondition	`event_log` must be a list
`write_gantt`	Precondition	`processors_count` must be a positive integer
`write_gantt`	Precondition	`path` must be a non-empty string

6.6 Design Rationale

Assertions serve as **executable documentation** of the contract each function expects. They allow bugs caused by incorrect inputs, such as a wrongly formatted file, a negative processor count, or a process being double-loaded into RAM, to be detected immediately at the point of failure. This is especially valuable in a simulation where a silent wrong value could propagate through many events before causing an observable error.

---

7. Team Contributions

Team Member	Phase 1	Phase 2	Phase 3	Phase 4
Sacara Samuel-Carlos	Implemented `models.py` (Process, Processor, SystemProcess), `gui.py`, `main.py` and the system process part of `simulator.py` (event queue, periodic system process release, `_try_run_syscall`)	Wrote `test_models.py` and `test_simulator.py` including the `FakeMemoryManager` mock	Added assertions in `simulator.py` and `models.py` (preconditions on constructor, event queue and process fields)	Wrote the Design and Architecture section of the documentation
Cucuteanu Lucian-Andrei	Implemented `memory.py` (LRU MemoryManager) and the virtual memory part of `simulator.py` (SWAP_IN/SWAP_OUT logic, disk queue, `_start_load`)	Wrote `test_memory.py`, covering both correct behaviour and invalid input scenarios	Added assertions in `memory.py` (preconditions on `evict_lru` and `complete_load`, postcondition on `used_ram`)	Wrote the Module Descriptions and Use of Assertions sections of the documentation
Dragos Gabriel-Catalin	Implemented `parser.py`, `output.py` and the user scheduling part of `simulator.py` (Round-Robin dispatch, processor affinity, `_schedule`, main `run` loop)	Wrote `test_parser.py` and `test_output.py`, including all invalid input tests for the parser	Added assertions in `parser.py` and `output.py` (preconditions and postconditions)	Wrote the User Manual and Testing sections and assembled the final documentation